

NAME

ioctl_userfaultfd – create a file descriptor for handling page faults in user space

LIBRARY

Standard C library (*libc*, *-lc*)

SYNOPSIS

```
#include <linux/userfaultfd.h> /* Definition of UFFD* constants */
```

```
#include <sys/ioctl.h>
```

```
int ioctl(int fd, int op, ...);
```

DESCRIPTION

Various **ioctl(2)** operations can be performed on a userfaultfd object (created by a call to **userfaultfd(2)**) using calls of the form:

```
ioctl(fd, op, argp);
```

In the above, *fd* is a file descriptor referring to a userfaultfd object, *op* is one of the operations listed below, and *argp* is a pointer to a data structure that is specific to *op*.

The various **ioctl(2)** operations are described below. The **UFFDIO_API**, **UFFDIO_REGISTER**, and **UFFDIO_UNREGISTER** operations are used to *configure* userfaultfd behavior. These operations allow the caller to choose what features will be enabled and what kinds of events will be delivered to the application. The remaining operations are *ange* operations. These operations enable the calling application to resolve page-fault events.

UFFDIO_API

(Since Linux 4.3.) Enable operation of the userfaultfd and perform API handshake.

The *argp* argument is a pointer to a *uffdio_api* structure, defined as:

```
struct uffdio_api {
    __u64 api;           /* Requested API version (input) */
    __u64 features;      /* Requested features (input/output) */
    __u64 ioctls;       /* Available ioctl() operations (output) */
};
```

The *api* field denotes the API version requested by the application. The kernel verifies that it can support the requested API version, and sets the *features* and *ioctls* fields to bit masks representing all the available features and the generic **ioctl(2)** operations available.

Since Linux 4.11, applications should use the *features* field to perform a two-step handshake. First, **UFFDIO_API** is called with the *features* field set to zero. The kernel responds by setting all supported feature bits.

Applications which do not require any specific features can begin using the userfaultfd immediately. Applications which do need specific features should call **UFFDIO_API** again with a subset of the reported feature bits set to enable those features.

Before Linux 4.11, the *features* field must be initialized to zero before the call to **UFFDIO_API**, and zero (i.e., no feature bits) is placed in the *features* field by the kernel upon return from **ioctl(2)**.

If the application sets unsupported feature bits, the kernel will zero out the returned *uffdio_api* structure and return **EINVAL**.

The following feature bits may be set:

UFFD_FEATURE_EVENT_FORK (since Linux 4.11)

When this feature is enabled, the userfaultfd objects associated with a parent process are duplicated into the child process during **fork(2)** and a **UFFD_EVENT_FORK** event is delivered to the userfaultfd monitor

UFFD_FEATURE_EVENT_REMAP (since Linux 4.11)

If this feature is enabled, when the faulting process invokes **mremap**(2), the userfaultfd monitor will receive an event of type **UFFD_EVENT_REMAP**.

UFFD_FEATURE_EVENT_REMOVE (since Linux 4.11)

If this feature is enabled, when the faulting process calls **madvise**(2) with the **MADV_DONTNEED** or **MADV_REMOVE** advice value to free a virtual memory area the userfaultfd monitor will receive an event of type **UFFD_EVENT_REMOVE**.

UFFD_FEATURE_EVENT_UNMAP (since Linux 4.11)

If this feature is enabled, when the faulting process unmaps virtual memory either explicitly with **munmap**(2), or implicitly during either **mmap**(2) or **mremap**(2), the userfaultfd monitor will receive an event of type **UFFD_EVENT_UNMAP**.

UFFD_FEATURE_MISSING_HUGETLBFS (since Linux 4.11)

If this feature bit is set, the kernel supports registering userfaultfd ranges on hugetlbfs virtual memory areas

UFFD_FEATURE_MISSING_SHMEM (since Linux 4.11)

If this feature bit is set, the kernel supports registering userfaultfd ranges on shared memory areas. This includes all kernel shared memory APIs: System V shared memory, **tmpfs**(5), shared mappings of */dev/zero*, **mmap**(2) with the **MAP_SHARED** flag set, **memfd_create**(2), and so on.

UFFD_FEATURE_SIGBUS (since Linux 4.14)

If this feature bit is set, no page-fault events (**UFFD_EVENT_PAGEFAULT**) will be delivered. Instead, a **SIGBUS** signal will be sent to the faulting process. Applications using this feature will not require the use of a userfaultfd monitor for processing memory accesses to the regions registered with userfaultfd.

UFFD_FEATURE_THREAD_ID (since Linux 4.14)

If this feature bit is set, *uffd_msg.pagefault.feat.ptid* will be set to the faulted thread ID for each page-fault message.

UFFD_FEATURE_PAGEFAULT_FLAG_WP (since Linux 5.10)

If this feature bit is set, userfaultfd supports write-protect faults for anonymous memory. (Note that shmem / hugetlbfs support is indicated by a separate feature.)

UFFD_FEATURE_MINOR_HUGETLBFS (since Linux 5.13)

If this feature bit is set, the kernel supports registering userfaultfd ranges in minor mode on hugetlbfs-backed memory areas.

UFFD_FEATURE_MINOR_SHMEM (since Linux 5.14)

If this feature bit is set, the kernel supports registering userfaultfd ranges in minor mode on shmem-backed memory areas.

UFFD_FEATURE_EXACT_ADDRESS (since Linux 5.18)

If this feature bit is set, *uffd_msg.pagefault.address* will be set to the exact page-fault address that was reported by the hardware, and will not mask the offset within the page. Note that old Linux versions might indicate the exact address as well, even though the feature bit is not set.

UFFD_FEATURE_WP_HUGETLBFS_SHMEM (since Linux 5.19)

If this feature bit is set, userfaultfd supports write-protect faults for hugetlbfs and shmem / tmpfs memory.

UFFD_FEATURE_WP_UNPOPULATED (since Linux 6.4)

If this feature bit is set, the kernel will handle anonymous memory the same way as file memory, by allowing the user to write-protect unpopulated page table entries.

UFFD_FEATURE_POISON (since Linux 6.6)

If this feature bit is set, the kernel supports resolving faults with the **UFFDIO_POISON** ioctl.

UFFD_FEATURE_WP_ASYNC (since Linux 6.7)

If this feature bit is set, the write protection faults would be asynchronously resolved by the kernel.

The returned *ioctls* field can contain the following bits:

1 << _UFFDIO_API

The **UFFDIO_API** operation is supported.

1 << _UFFDIO_REGISTER

The **UFFDIO_REGISTER** operation is supported.

1 << _UFFDIO_UNREGISTER

The **UFFDIO_UNREGISTER** operation is supported.

This **ioctl(2)** operation returns 0 on success. On error, -1 is returned and *errno* is set to indicate the error. If an error occurs, the kernel may zero the provided *uffdio_api* structure. The caller should treat its contents as unspecified, and reinitialize it before re-attempting another **UFFDIO_API** call. Possible errors include:

EFAULT

argp refers to an address that is outside the calling process's accessible address space.

EINVAL

The API version requested in the *api* field is not supported by this kernel, or the *features* field passed to the kernel includes feature bits that are not supported by the current kernel version.

EINVAL

A previous **UFFDIO_API** call already enabled one or more features for this userfaultfd. Calling **UFFDIO_API** twice, the first time with no features set, is explicitly allowed as per the two-step feature detection handshake.

EPERM

The **UFFD_FEATURE_EVENT_FORK** feature was enabled, but the calling process doesn't have the **CAP_SYS_PTRACE** capability.

UFFDIO_REGISTER

(Since Linux 4.3.) Register a memory address range with the userfaultfd object. The pages in the range must be "compatible". Please refer to the list of register modes below for the compatible memory backends for each mode.

The *argp* argument is a pointer to a *uffdio_register* structure, defined as:

```
struct uffdio_range {
    __u64 start;    /* Start of range */
    __u64 len;      /* Length of range (bytes) */
};

struct uffdio_register {
    struct uffdio_range range;
    __u64 mode;     /* Desired mode of operation (input) */
    __u64 ioctls;   /* Available ioctl() operations (output) */
};
```

The *range* field defines a memory range starting at *start* and continuing for *len* bytes that should be handled by the userfaultfd.

The *mode* field defines the mode of operation desired for this memory region. The following values may be bitwise ORed to set the userfaultfd mode for the specified range:

UFFDIO_REGISTER_MODE_MISSING

Track page faults on missing pages. Since Linux 4.3, only private anonymous ranges are compatible. Since Linux 4.11, hugetlbfs and shared memory ranges are also compatible.

UFFDIO_REGISTER_MODE_WP

Track page faults on write-protected pages. Since Linux 5.7, only private anonymous ranges are compatible.

UFFDIO_REGISTER_MODE_MINOR

Track minor page faults. Since Linux 5.13, only hugetlbfs ranges are compatible. Since Linux 5.14, compatibility with shmem ranges was added.

If the operation is successful, the kernel modifies the *ioctls* bit-mask field to indicate which **ioctl(2)** operations are available for the specified range. This returned bit mask can contain the following bits:

1 << _UFFDIO_COPY

The **UFFDIO_COPY** operation is supported.

1 << _UFFDIO_WAKE

The **UFFDIO_WAKE** operation is supported.

1 << _UFFDIO_WRITEPROTECT

The **UFFDIO_WRITEPROTECT** operation is supported.

1 << _UFFDIO_ZEROPAGE

The **UFFDIO_ZEROPAGE** operation is supported.

1 << _UFFDIO_CONTINUE

The **UFFDIO_CONTINUE** operation is supported.

1 << _UFFDIO_POISON

The **UFFDIO_POISON** operation is supported.

This **ioctl(2)** operation returns 0 on success. On error, -1 is returned and *errno* is set to indicate the error. Possible errors include:

EBUSY

A mapping in the specified range is registered with another userfaultfd object.

EFAULT

argp refers to an address that is outside the calling process's accessible address space.

EINVAL

An invalid or unsupported bit was specified in the *mode* field; or the *mode* field was zero.

EINVAL

There is no mapping in the specified address range.

EINVAL

range.start or *range.len* is not a multiple of the system page size; or, *range.len* is zero; or these fields are otherwise invalid.

EINVAL

There as an incompatible mapping in the specified address range.

UFFDIO_UNREGISTER

(Since Linux 4.3.) Unregister a memory address range from userfaultfd. The pages in the range must be "compatible" (see the description of **UFFDIO_REGISTER**.)

The address range to unregister is specified in the *uffdio_range* structure pointed to by *argp*.

This **ioctl(2)** operation returns 0 on success. On error, -1 is returned and *errno* is set to indicate the error. Possible errors include:

EINVAL

Either the *start* or the *len* field of the *uffdio_range* structure was not a multiple of the system page size; or the *len* field was zero; or these fields were otherwise invalid.

EINVAL

There as an incompatible mapping in the specified address range.

EINVAL

There was no mapping in the specified address range.

UFFDIO_COPY

(Since Linux 4.3.) Atomically copy a continuous memory chunk into the userfault registered range and optionally wake up the blocked thread. The source and destination addresses and the number of bytes to copy are specified by the *src*, *dst*, and *len* fields of the *uffdio_copy* structure pointed to by *argp*:

```
struct uffdio_copy {
    __u64 dst;      /* Destination of copy */
    __u64 src;      /* Source of copy */
    __u64 len;      /* Number of bytes to copy */
    __u64 mode;     /* Flags controlling behavior of copy */
    __s64 copy;     /* Number of bytes copied, or negated error */
};
```

The following value may be bitwise ORed in *mode* to change the behavior of the **UFFDIO_COPY** operation:

UFFDIO_COPY_MODE_DONTWAKE

Do not wake up the thread that waits for page-fault resolution

UFFDIO_COPY_MODE_WP

Copy the page with read-only permission. This allows the user to trap the next write to the page, which will block and generate another write-protect userfault message. This is used only when both **UFFDIO_REGISTER_MODE_MISSING** and **UFFDIO_REGISTER_MODE_WP** modes are enabled for the registered range.

The *copy* field is used by the kernel to return the number of bytes that was actually copied, or an error (a negated *errno*-style value). If the value returned in *copy* doesn't match the value that was specified in *len*, the operation fails with the error **EAGAIN**. The *copy* field is output-only; it is not read by the **UFFDIO_COPY** operation.

This **ioctl(2)** operation returns 0 on success. In this case, the entire area was copied. On error, -1 is returned and *errno* is set to indicate the error. Possible errors include:

EAGAIN

The number of bytes copied (i.e., the value returned in the *copy* field) does not equal the value that was specified in the *len* field.

EINVAL

Either *dst* or *len* was not a multiple of the system page size, or the range specified by *src* and *len* or *dst* and *len* was invalid.

EINVAL

An invalid bit was specified in the *mode* field.

ENOENT (since Linux 4.11)

The faulting process has changed its virtual memory layout simultaneously with an outstanding **UFFDIO_COPY** operation.

ENOSPC (from Linux 4.11 until Linux 4.13)

The faulting process has exited at the time of a **UFFDIO_COPY** operation.

ESRCH (since Linux 4.13)

The faulting process has exited at the time of a **UFFDIO_COPY** operation.

UFFDIO_ZEROPAGE

(Since Linux 4.3.) Zero out a memory range registered with userfaultfd.

The requested range is specified by the *range* field of the *uffdio_zeropage* structure pointed to by *argp*:

```

struct uffdio_zeropage {
    struct uffdio_range range;
    __u64 mode;          /* Flags controlling behavior of copy */
    __s64 zeropage; /* Number of bytes zeroed, or negated error */
};

```

The following value may be bitwise ORed in *mode* to change the behavior of the **UFFDIO_ZEROPAGE** operation:

UFFDIO_ZEROPAGE_MODE_DONTWAKE

Do not wake up the thread that waits for page-fault resolution.

The *zeropage* field is used by the kernel to return the number of bytes that was actually zeroed, or an error in the same manner as **UFFDIO_COPY**. If the value returned in the *zeropage* field doesn't match the value that was specified in *range.len*, the operation fails with the error **EAGAIN**. The *zeropage* field is output-only; it is not read by the **UFFDIO_ZEROPAGE** operation.

This **ioctl**(2) operation returns 0 on success. In this case, the entire area was zeroed. On error, -1 is returned and *errno* is set to indicate the error. Possible errors include:

EAGAIN

The number of bytes zeroed (i.e., the value returned in the *zeropage* field) does not equal the value that was specified in the *range.len* field.

EINVAL

Either *range.start* or *range.len* was not a multiple of the system page size; or *range.len* was zero; or the range specified was invalid.

EINVAL

An invalid bit was specified in the *mode* field.

ESRCH (since Linux 4.13)

The faulting process has exited at the time of a **UFFDIO_ZEROPAGE** operation.

UFFDIO_WAKE

(Since Linux 4.3.) Wake up the thread waiting for page-fault resolution on a specified memory address range.

The **UFFDIO_WAKE** operation is used in conjunction with **UFFDIO_COPY** and **UFFDIO_ZEROPAGE** operations that have the **UFFDIO_COPY_MODE_DONTWAKE** or **UFFDIO_ZEROPAGE_MODE_DONTWAKE** bit set in the *mode* field. The userfault monitor can perform several **UFFDIO_COPY** and **UFFDIO_ZEROPAGE** operations in a batch and then explicitly wake up the faulting thread using **UFFDIO_WAKE**.

The *argp* argument is a pointer to a *uffdio_range* structure (shown above) that specifies the address range.

This **ioctl**(2) operation returns 0 on success. On error, -1 is returned and *errno* is set to indicate the error. Possible errors include:

EINVAL

The *start* or the *len* field of the *uffdio_range* structure was not a multiple of the system page size; or *len* was zero; or the specified range was otherwise invalid.

UFFDIO_WRITEPROTECT

(Since Linux 5.7.) Write-protect or write-unprotect a userfaultfd-registered memory range registered with mode **UFFDIO_REGISTER_MODE_WP**.

The *argp* argument is a pointer to a *uffdio_range* structure as shown below:

```

struct uffdio_writeprotect {
    struct uffdio_range range; /* Range to change write permission */
    __u64 mode;               /* Mode to change write permission */
};

```

There are two mode bits that are supported in this structure:

UFFDIO_WRITEPROTECT_MODE_WP

When this mode bit is set, the `ioctl` will be a write-protect operation upon the memory range specified by *range*. Otherwise it will be a write-unprotect operation upon the specified range, which can be used to resolve a userfaultfd write-protect page fault.

UFFDIO_WRITEPROTECT_MODE_DONTWAKE

When this mode bit is set, do not wake up any thread that waits for page-fault resolution after the operation. This can be specified only if **UFFDIO_WRITEPROTECT_MODE_WP** is not specified.

This **ioctl**(2) operation returns 0 on success. On error, -1 is returned and *errno* is set to indicate the error. Possible errors include:

EINVAL

The *start* or the *len* field of the *uffdio_range* structure was not a multiple of the system page size; or *len* was zero; or the specified range was otherwise invalid.

EAGAIN

The process was interrupted; retry this call.

ENOENT

The range specified in *range* is not valid. For example, the virtual address does not exist, or not registered with userfaultfd write-protect mode.

EFAULT

Encountered a generic fault during processing.

UFFDIO_CONTINUE

(Since Linux 5.13.) Resolve a minor page fault by installing page table entries for existing pages in the page cache.

The *argp* argument is a pointer to a *uffdio_continue* structure as shown below:

```
struct uffdio_continue {
    struct uffdio_range range;
    /* Range to install PTEs for and continue */
    __u64 mode; /* Flags controlling the behavior of continue */
    __s64 mapped; /* Number of bytes mapped, or negated error */
};
```

The following value may be bitwise ORed in *mode* to change the behavior of the **UFFDIO_CONTINUE** operation:

UFFDIO_CONTINUE_MODE_DONTWAKE

Do not wake up the thread that waits for page-fault resolution.

The *mapped* field is used by the kernel to return the number of bytes that were actually mapped, or an error in the same manner as **UFFDIO_COPY**. If the value returned in the *mapped* field doesn't match the value that was specified in *range.len*, the operation fails with the error **EAGAIN**. The *mapped* field is output-only; it is not read by the **UFFDIO_CONTINUE** operation.

This **ioctl**(2) operation returns 0 on success. In this case, the entire area was mapped. On error, -1 is returned and *errno* is set to indicate the error. Possible errors include:

EAGAIN

The number of bytes mapped (i.e., the value returned in the *mapped* field) does not equal the value that was specified in the *range.len* field.

EEXIST

One or more pages were already mapped in the given range.

EFAULT

No existing page could be found in the page cache for the given range.

EINVAL

Either *range.start* or *range.len* was not a multiple of the system page size; or *range.len* was zero; or the range specified was invalid.

EINVAL

An invalid bit was specified in the *mode* field.

ENOENT

The faulting process has changed its virtual memory layout simultaneously with an outstanding **UFFDIO_CONTINUE** operation.

ENOMEM

Allocating memory needed to setup the page table mappings failed.

ESRCH

The faulting process has exited at the time of a **UFFDIO_CONTINUE** operation.

UFFDIO_POISON

(Since Linux 6.6.) Mark an address range as "poisoned". Future accesses to these addresses will raise a **SIGBUS** signal. Unlike **MADV_HWPOISON** this works by installing page table entries, rather than "really" poisoning the underlying physical pages. This means it only affects this particular address space.

The *argp* argument is a pointer to a *uffdio_poison* structure as shown below:

```
struct uffdio_poison {
    struct uffdio_range range;
                                /* Range to install poison PTE markers in */
    __u64 mode;                 /* Flags controlling the behavior of poison */
    __s64 updated;              /* Number of bytes poisoned, or negated error */
};
```

The following value may be bitwise ORed in *mode* to change the behavior of the **UFFDIO_POISON** operation:

UFFDIO_POISON_MODE_DONTWAKE

Do not wake up the thread that waits for page-fault resolution.

The *updated* field is used by the kernel to return the number of bytes that were actually poisoned, or an error in the same manner as **UFFDIO_COPY**. If the value returned in the *updated* field doesn't match the value that was specified in *range.len*, the operation fails with the error **EAGAIN**. The *updated* field is out-put-only; it is not read by the **UFFDIO_POISON** operation.

This **ioctl**(2) operation returns 0 on success. In this case, the entire area was poisoned. On error, -1 is returned and *errno* is set to indicate the error. Possible errors include:

EAGAIN

The number of bytes mapped (i.e., the value returned in the *updated* field) does not equal the value that was specified in the *range.len* field.

EINVAL

Either *range.start* or *range.len* was not a multiple of the system page size; or *range.len* was zero; or the range specified was invalid.

EINVAL

An invalid bit was specified in the *mode* field.

EEXIST

One or more pages were already mapped in the given range.

ENOENT

The faulting process has changed its virtual memory layout simultaneously with an outstanding **UFFDIO_POISON** operation.

ENOMEM

Allocating memory for page table entries failed.

ESRCH

The faulting process has exited at the time of a **UFFDIO_POISON** operation.

RETURN VALUE

See descriptions of the individual operations, above.

ERRORS

See descriptions of the individual operations, above. In addition, the following general errors can occur for all of the operations described above:

EFAULT

argp does not point to a valid memory address.

EINVAL

(For all operations except **UFFDIO_API**.) The userfaultfd object has not yet been enabled (via the **UFFDIO_API** operation).

STANDARDS

Linux.

BUGS

In order to detect available userfault features and enable some subset of those features the userfaultfd file descriptor must be closed after the first **UFFDIO_API** operation that queries features availability and re-opened before the second **UFFDIO_API** operation that actually enables the desired features.

EXAMPLES

See **userfaultfd(2)**.

SEE ALSO

ioctl(2), **mmap(2)**, **userfaultfd(2)**

Documentation/admin-guide/mm/userfaultfd.rst in the Linux kernel source tree